Ejercicios que comenzamos a analizar en la clase del 8 de mayo, 2025

Ejercicio 15 (supernetting). Dada la siguiente tabla de ruteo basada en clases, reescríbala utilizando CIDR intentando minimizar lo más posible el número de entradas:

212.128.175.0	15.0.0.1
212.128.176.0	15.0.0.1
212.128.177.0	15.0.0.1
212.128.178.0	15.0.0.1
212.128.179.0	15.0.0.1
212.128.180.0	15.0.0.1
212.128.181.0	15.0.0.1
212.128.182.0	15.0.0.1
212.128.183.0	15.0.0.1
212.128.184.0	15.0.0.1
212.128.185.0	15.0.0.1

Interpretación del enunciado. Observamos que el enunciado no especifica máscaras de red, con lo cual no tenemos forma de saber si se trata de direcciones de red o direcciones de host.

Teniendo en cuenta que generalmente las direcciones de red se escriben con los bits de la parte de host en 0 (nulos), vamos a asumir entonces que todas las direcciones de la primera columna son direcciones de red /24. Tiene sentido porque el enunciado nos dice que la tabla de ruteo está basada en clases: vamos a asumir que la columna de la izquierda está compuesta por redes de la clase “C”.

Para la columna de la derecha vamos a asumir que es la dirección del próximo salto (también llamado campo next-hop, destino o gateway de la tabla de ruteo). Es decir, asumimos que son direcciones de host (*).

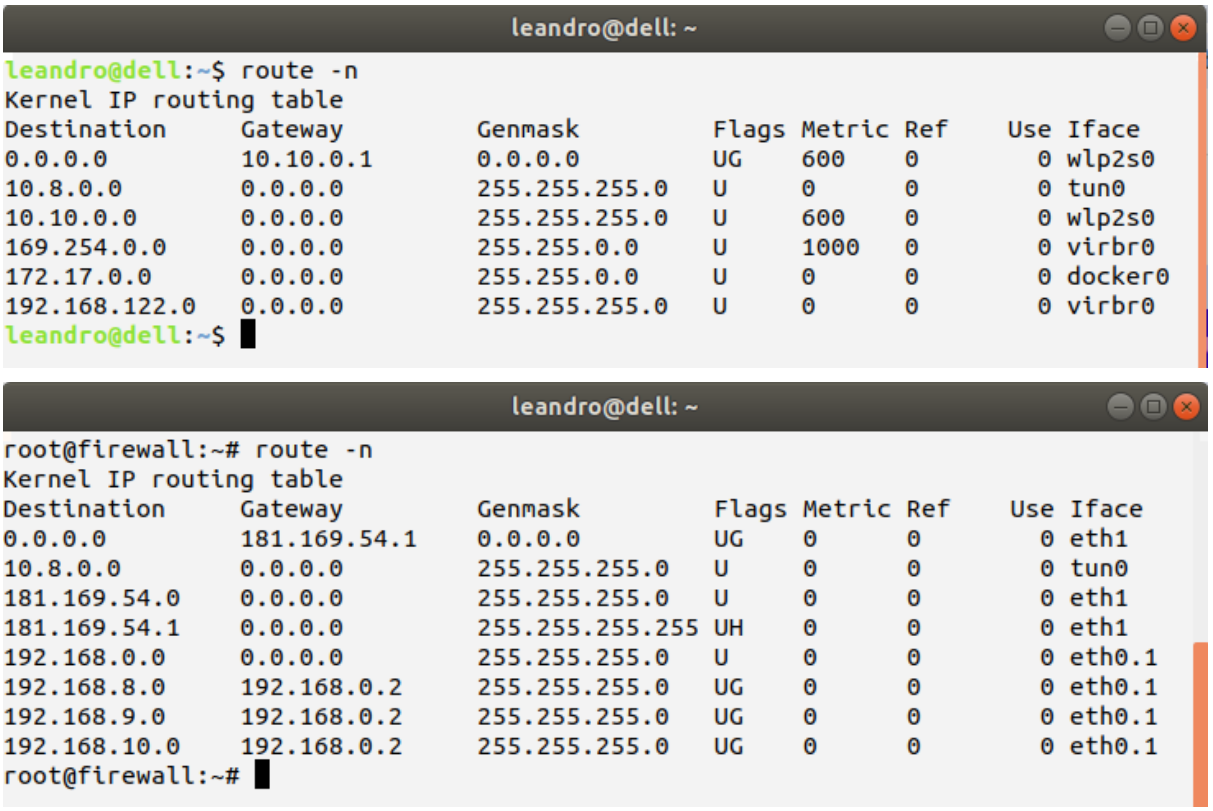
Sumarización o supernetting. Vamos a escribirlas de una forma que nos permita agruparlas en función de patrones en los prefijos que compartan un mismo next-hop. Entonces:

```
175 = 128 + 32 +    + 8 + 4 + 2 +   = 1010 1111
176 = 128 + 32 + 16                 = 1011 0000
...
183 = 128 + 32 + 16     + 4 + 2 + 1 = 1011 0111
184 = 128 + 32 + 16 + 8             = 1011 1000
185 = 128 + 32 + 16 + 8         + 1 = 1011 1001
```

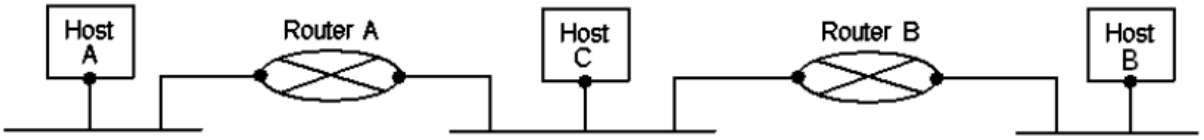
Luego,

Destino	Next-hop o gateway
212.128.175.0/24	15.0.0.1
212.128.176.0/21	15.0.0.1
212.128.184.0/23	15.0.0.1

(*) Comportamiento por defecto para mostrar el next-hop en algunas implementaciones.



Ejercicio 11. En el siguiente esquema de red, el host A envía un ping al host B pero el mismo se encuentra apagado. Suponiendo que todos los routers y hosts tienen activados sus protocolos ICMP sin filtrar, indicar si el ping se realiza con éxito o qué sucede en caso contrario.



Para responder esta pregunta, veamos qué dice la sección 5.6 del libro de Kurose:

5.6 ICMP: The Internet Control Message Protocol

The Internet Control Message Protocol (ICMP), specified in [RFC 792], is used by hosts and routers to communicate network-layer information to each other. The most typical use of ICMP is for error reporting. For example, when running an HTTP session, you may have encountered an error message such as "Destination network unreachable." This message had its origins in ICMP. At some point, an IP router was unable to find a path to the host specified in your HTTP request. That router created and sent an ICMP message to your host indicating the error.

ICMP is often considered part of IP, but architecturally it lies just above IP, as ICMP messages are carried inside IP datagrams. That is, ICMP messages are carried

Veamos también que dice en la RFC que describe el mensaje del protocolo ICMP:

```
Description

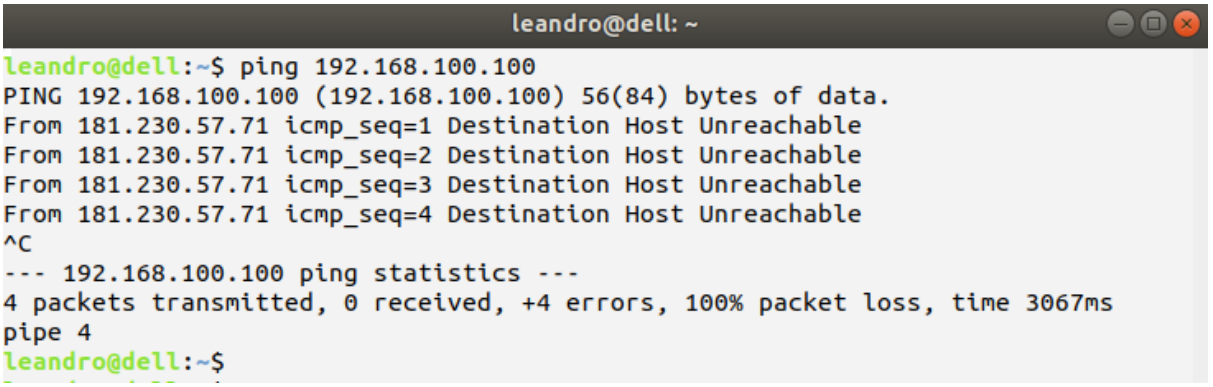
      If, according to the information in the gateway's routing tables,
      the network specified in the internet destination field of a
      datagram is unreachable, e.g., the distance to the network is
      infinity, the gateway may send a destination unreachable message
      to the internet source host of the datagram.  In addition, in some
      networks, the gateway may be able to determine if the internet
      destination host is unreachable.  Gateways in these networks may
      send destination unreachable messages to the source host when the
      destination host is unreachable.

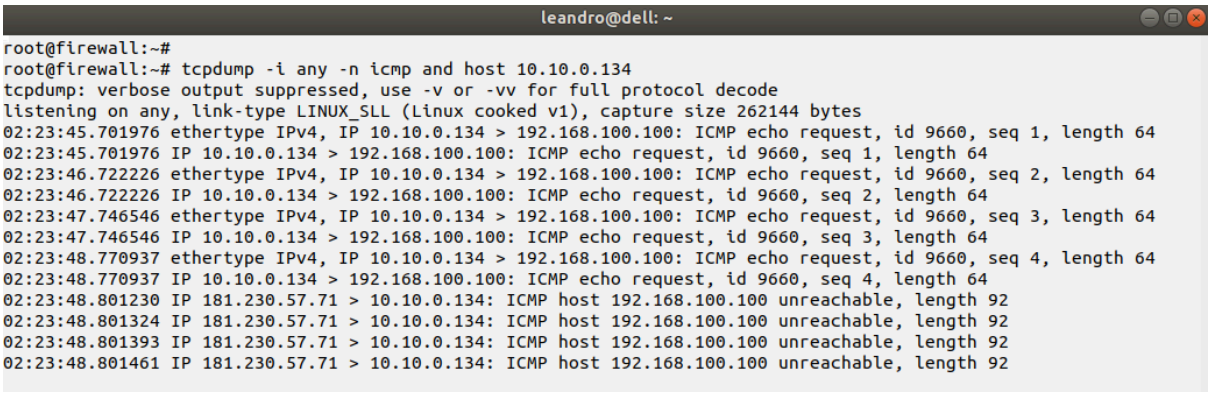
      If, in the destination host, the IP module cannot deliver the
      datagram  because the indicated protocol module or process port is
      not active, the destination host may send a destination
      unreachable message to the source host.

      Another case is when a datagram must be fragmented to be forwarded
      by a gateway yet the Don't Fragment flag is on.  In this case the
      gateway must discard the datagram and may return a destination
      unreachable message.

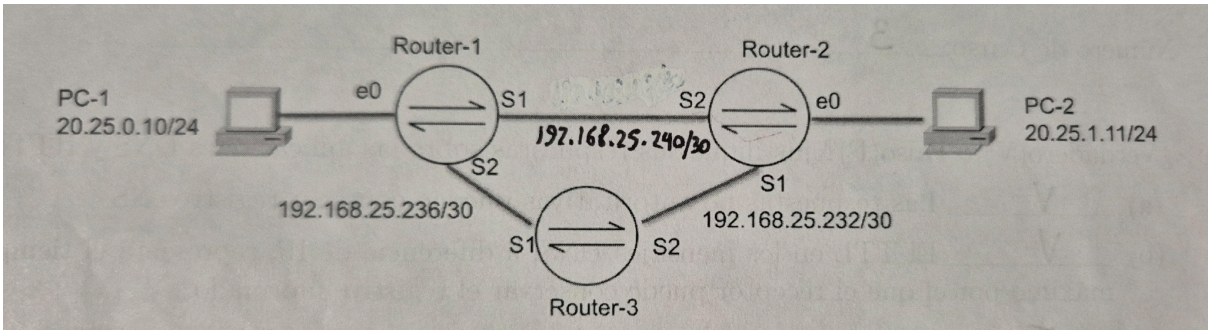
      Codes 0, 1, 4, and 5 may be received from a gateway.  Codes 2 and
      3 may be received from a host.
```

El ping no se realiza con éxito porque el host B está apagado por hipótesis. Entonces, según vimos arriba, el router “B” podría enviar un mensaje ICMP tipo 3 (destination unreachable message), código 1 (host unreachable).





Ejercicio 4 (parcial segundo cuatrimestre, 2024). Dada la siguiente topología de red:



Router-1		
subred/másc	p.salto	int
20.25.0.0/24	-	e0
192.168.25.236/30	-	S2
20.25.1.0/24	(x1)	S1

Router-2		
subred/másc	p.salto	int
20.25.1.0/24	-	e0
192.168.25.232/30	-	S1
0.0.0.0/0	(x2)	S1

(a) Completar la configuración mínima de nivel de red en todas las interfaces. Si necesita definir más subredes, utilice otras de la misma clase privada, ajustando la cantidad de interfaces requeridas.

Asignación de direcciones IP y máscaras de red para cada interfaz:

Router-1	e0	20.25.0.1/24
	s1	192.168.25.241/30
	s2	192.168.25.237/30
Router-2	e0	20.25.1.1/24
	s1	192.168.25.233/30
	s2	192.168.25.242/30
Router-3	s1	192.168.25.238/30
	s2	192.168.25.234/30

(b) Completar los elementos x1 y x2 de las tablas de ruteo. Si en el punto anterior se definieron subredes, agregarlas donde corresponda. Especificar la tabla mínima de Router-3.

Para los puntos de salto (también llamados gateways o next-hops en el ámbito de la tabla de ruteo) vamos a configurar las siguientes direcciones de host:

x1 ~ 192.168.25.242
x2 ~ 192.168.25.234

Para configurar la tabla mínima del Router-3 vamos tener en cuenta algunas características deseables para una tabla de ruteo:

- Contiene las rutas locales a todas las redes directamente conectadas: en nuestro caso, las 192.168.25.236/30 y 192.168.25.232/30.
- Vamos a incluir una ruta por defecto (la 0.0.0.0/0) que asegure forwardear datagramas para cualquier otro prefijo de red que no esté explícitamente configurado en la misma.
- Además agregamos una ruta para que la 20.25.0.0/24 pueda alcanzarse directamente a través de R1 (así evitamos dar la vueltita completa por la ruta default del punto anterior).

Router-3		
subred/máscara	p. salto	interfaz
192.168.25.236/30	local	S1
192.168.25.232/30	local	S2
20.25.0.0/24	192.168.25.237	S1
0.0.0.0/0	192.168.25.233	S2

Obs: Se deja como inquietud agregar la 192.168.25.240/30 a las tablas de ruteo del Router-1 y Router-2.

Obs: notar que x1 y x2 deben ser direcciones de host. No podrían ser redes (por ejemplo no deberíamos fijarlos a redes /30) ya que no permitiría que el router pueda forwardear los datagramas a destino.

(c) Suponer la ejecución de un `traceroute` desde la PC-1 a la PC-2. Indicar en una tabla: el recorrido de cada datagrama, direcciones origen y destino y salida por pantalla.

datagrama	recorrido	dirección origen	dirección destino
1	PC1 - R1	20.25.0.10	20.25.1.11
2	R1 - PC1	20.25.0.1	20.25.0.10
3	PC1 - R1 - R2	20.25.0.10	20.25.1.11
4	R2 - R3 - R1 - PC1	192.168.25.242	20.25.0.10
5	PC1 - R1 - R2 - PC2	20.25.0.10	20.25.1.11
6	PC2 - R2 - R3 - R1 - PC1	20.25.1.11	20.25.0.10

La salida será:

```
1 20.25.0.1
2 192.168.25.242
3 20.25.1.11
```

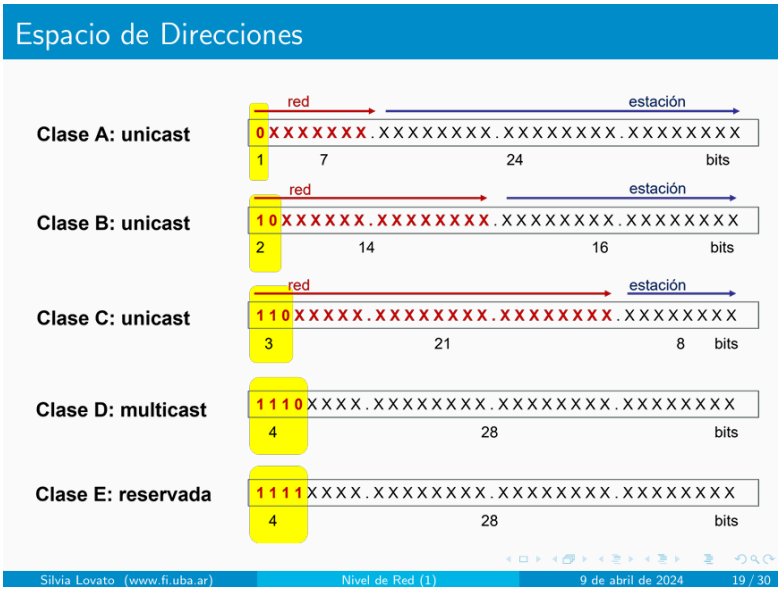
16. Resolver: En la tabla de ruteo siguiente indique la entrada que tendrán correspondencia con la dirección destino 128.9.200.20. Por qué interfaz será transmitido un datagrama que tenga esa dirección destino. Explicar.

128.0.0.0/8	3
128.9.0.0/16	5
128.9.192.0/20	2
128.9.192.0/22	4
128.9.192.0/24	7
128.9.200.0/24	8
128.9.200.0/28	10
128.9.200.16/28	1
128.9.200.16/30	9
128.9.192.8/30	6

Como estamos ante una tabla de ruteo que tiene dirección y longitud de máscara entonces estamos en el escenario CIDR (Classless Interdomain Routing). En los inicios de IP el escenario era el de Classful Routing (RFC 791, circa 1981).

The Internet's address assignment strategy is known as **Classless Interdomain Routing** (**CIDR**—pronounced *cider*) [RFC 4632]. CIDR generalizes the notion of subnet addressing. As with subnet addressing, the 32-bit IP address is divided into two parts and again has the dotted-decimal form *a.b.c.d/x*, where *x* indicates the number of bits in the first part of the address.

The *x* most significant bits of an address of the form *a.b.c.d/x* constitute the network portion of the IP address, and are often referred to as the **prefix** (or *network prefix*) of the address. An organization is typically assigned a block of contiguous



Por lo tanto cuando se usa CIDR para decidir por cuál ruta salir hay que basarse en el principio longest-match tal como vemos en la RFC 1338

```
4.1.  Protocol-independent semantic changes
   There are two fundamental changes which must be applied to Inter-
   Domain routing protocols in order for this plan to work. First, the
   concept of network "class" needs to be deprecated - this plan assumes
   that routing destinations are represented by network+mask pairs and
   that routing is done on a longest-match basis (i.e., for a given
   destination which matches multiple network+mask pairs, the match with
   the longest mask is used).
```

El matcheo en CIDR se hace de la siguiente manera:

match = (dst_IP_dgram & netmask) == dst_net

para dar un ejemplo tomaremosla primer fila de la tabla de ruteo

```
match = ( 128.9.200.20 &  255.0.0.0 ) == 128.0.0.0
match = (          128.0.0.0        ) == 128.0.0.0 = true
```

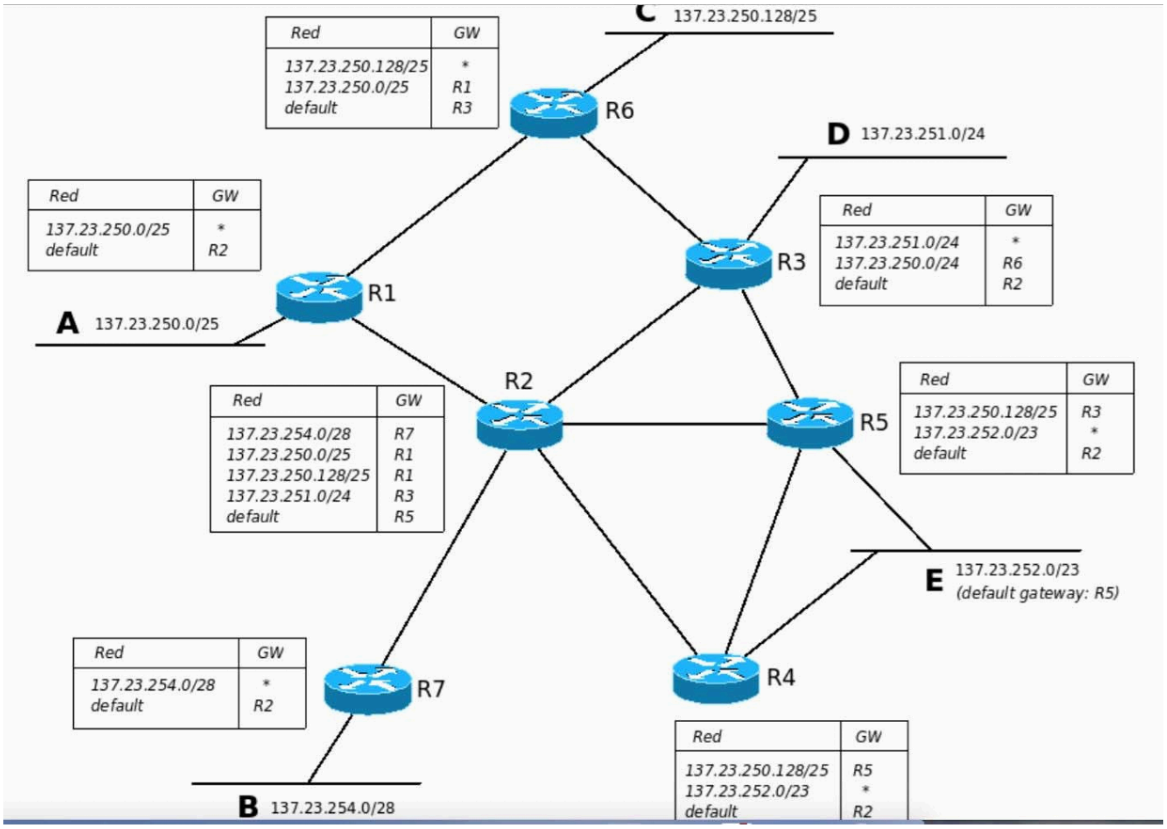
Entonces siguiendo la premisa de CIDR, debemos evaluar las máscaras más largas (selectivas) primero. Cuando matcheamos dejamos de evaluar, ya que habrá sido el de la máscara más larga. Entonces en este caso se forwardea el datagrama por la interfaz 1.

Dst_net/mask	IF	128.9.200.20 match?
128.9.192.8/30	6	no
128.9.200.16/30	9	no
128.9.200.16/28	1	si
128.9.200.0/28	10	
128.9.200.0/24	8	
128.9.192.0/24	7	
128.9.192.0/22	4	
128.9.192.0/20	2	
128.9.0.0/16	5	
128.0.0.0/8	3	

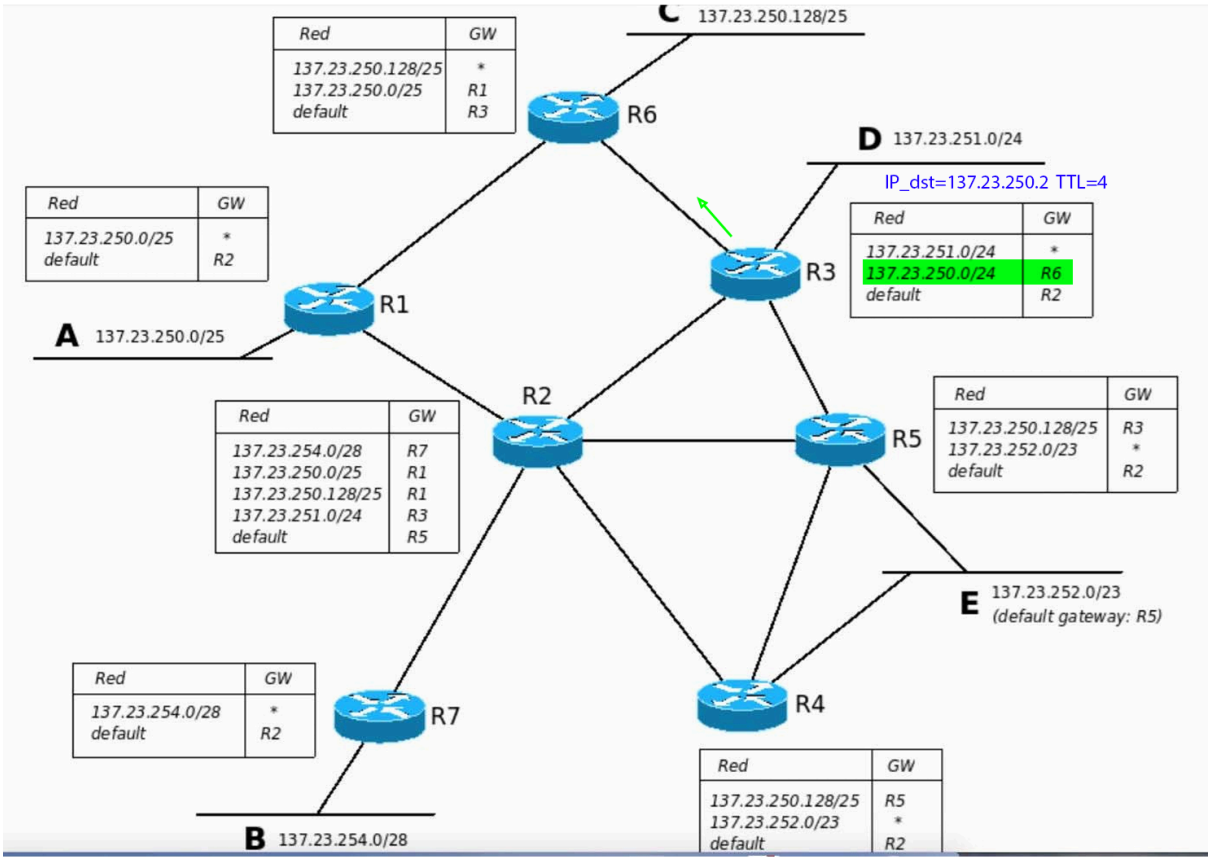
Si evaluaremos la cuenta del matcheo para todas las entradas veríamos que match para más de una entrada, por eso el orden de evaluación (longest-match first) es fundamental

Dst_net/mask	IF	128.9.200.20 match?
128.9.192.8/30	6	no
128.9.200.16/30	9	no
128.9.200.16/28	1	si
128.9.200.0/28	10	no
128.9.200.0/24	8	si
128.9.192.0/24	7	no
128.9.192.0/22	4	no
128.9.192.0/20	2	si
128.9.0.0/16	5	si
128.0.0.0/8	3	si

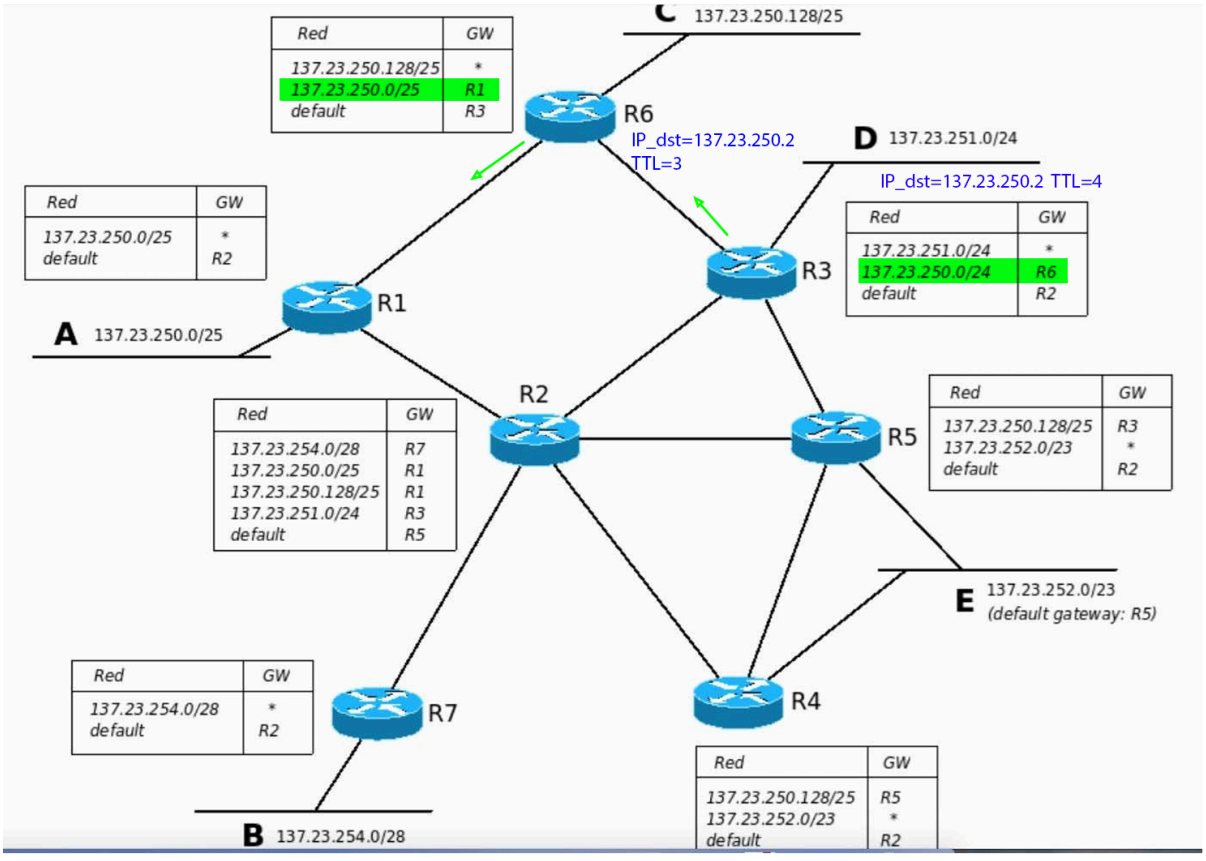
Ejercicio 28) Sea la red de la figura. Si un host en D hace un ping a un host en A configurando el ping con un solo mensaje y TTL = 4 indique todos los datagramas que se generarán como producto de ese ping y sus recorridos. Indicar los campos relevantes de cada uno de ellos.



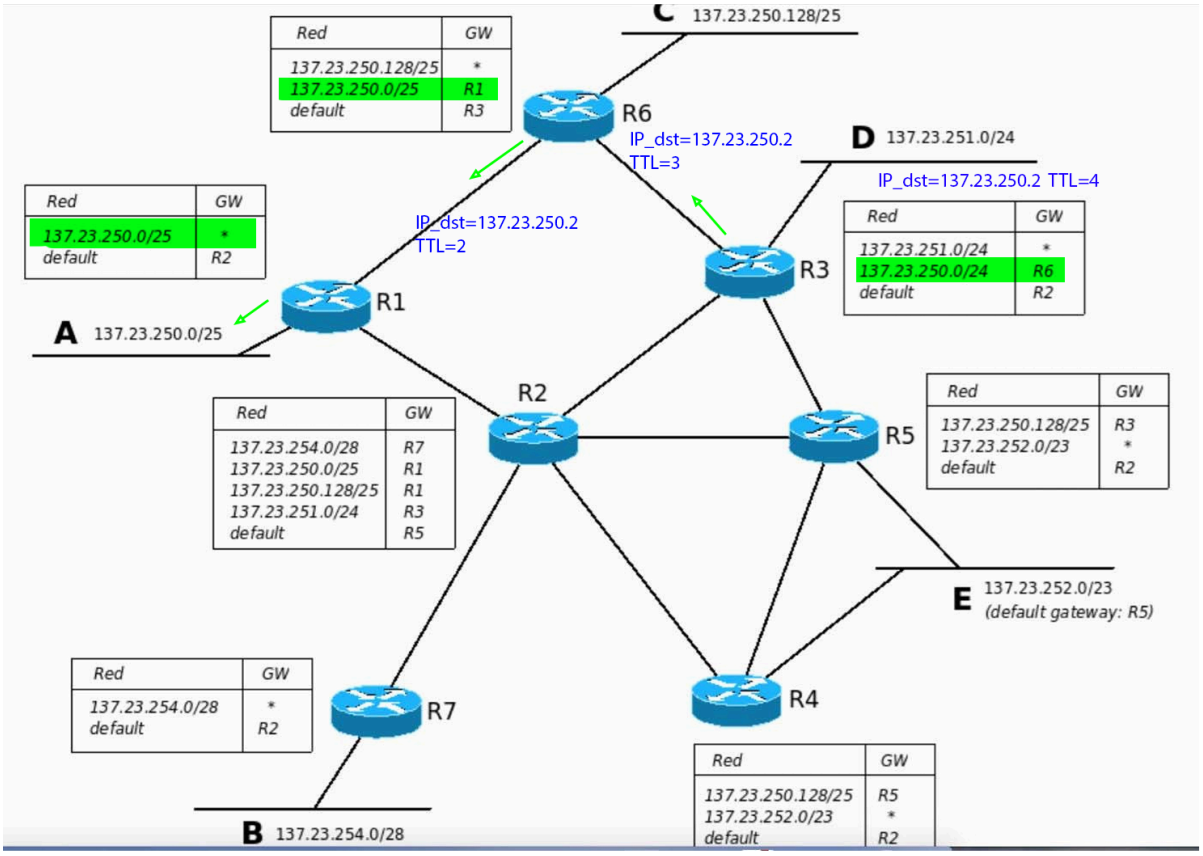
Paso 1

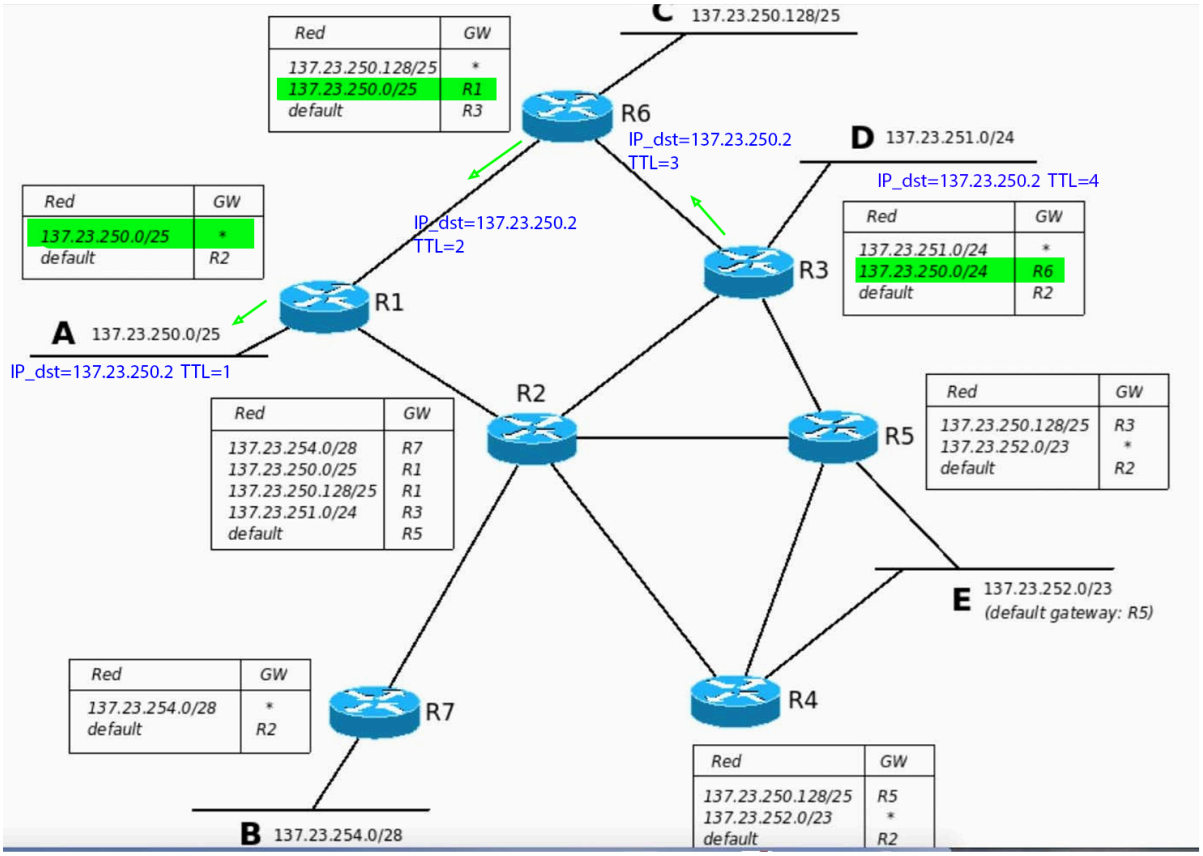

C 137.23.250.128/25
R6
D 137.23.251.0/24
IP_dst=137.23.250.2 TTL=4
R3
R1
A 137.23.250.0/25
R2
R5
E 137.23.252.0/23
(default gateway: R5)
R7
R4
B 137.23.254.0/28
Red | GW
137.23.250.128/25 | *
137.23.250.0/25 | R1
default | R3
Red | GW
137.23.250.0/25 | *
default | R2
Red | GW
137.23.251.0/24 | *
137.23.250.0/24 | R6
default | R2
Red | GW
137.23.254.0/28 | R7
137.23.250.0/25 | R1
137.23.250.128/25 | R1
137.23.251.0/24 | R3
default | R5
Red | GW
137.23.250.128/25 | R3
137.23.252.0/23 | *
default | R2
Red | GW
137.23.254.0/28 | *
default | R2
Red | GW
137.23.250.128/25 | R5
137.23.252.0/23 | *
default | R2

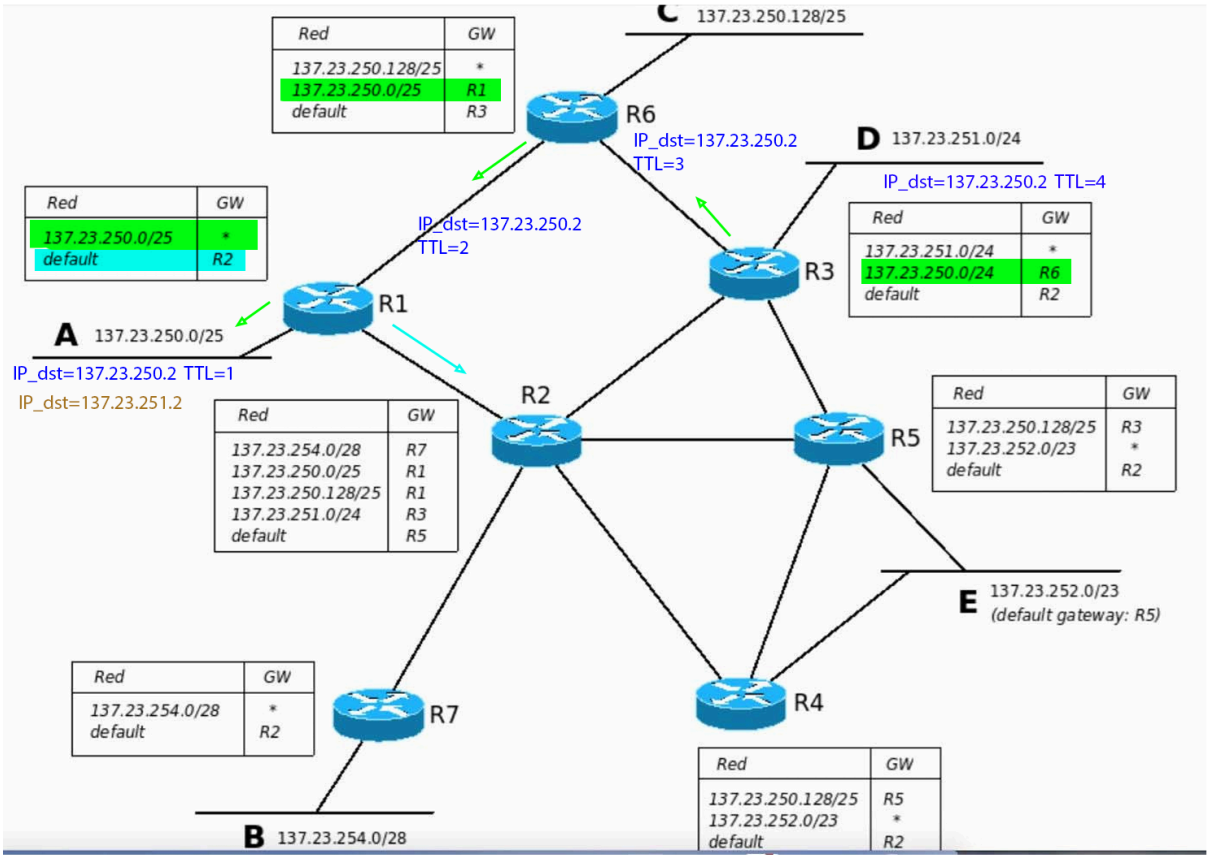
Paso 2

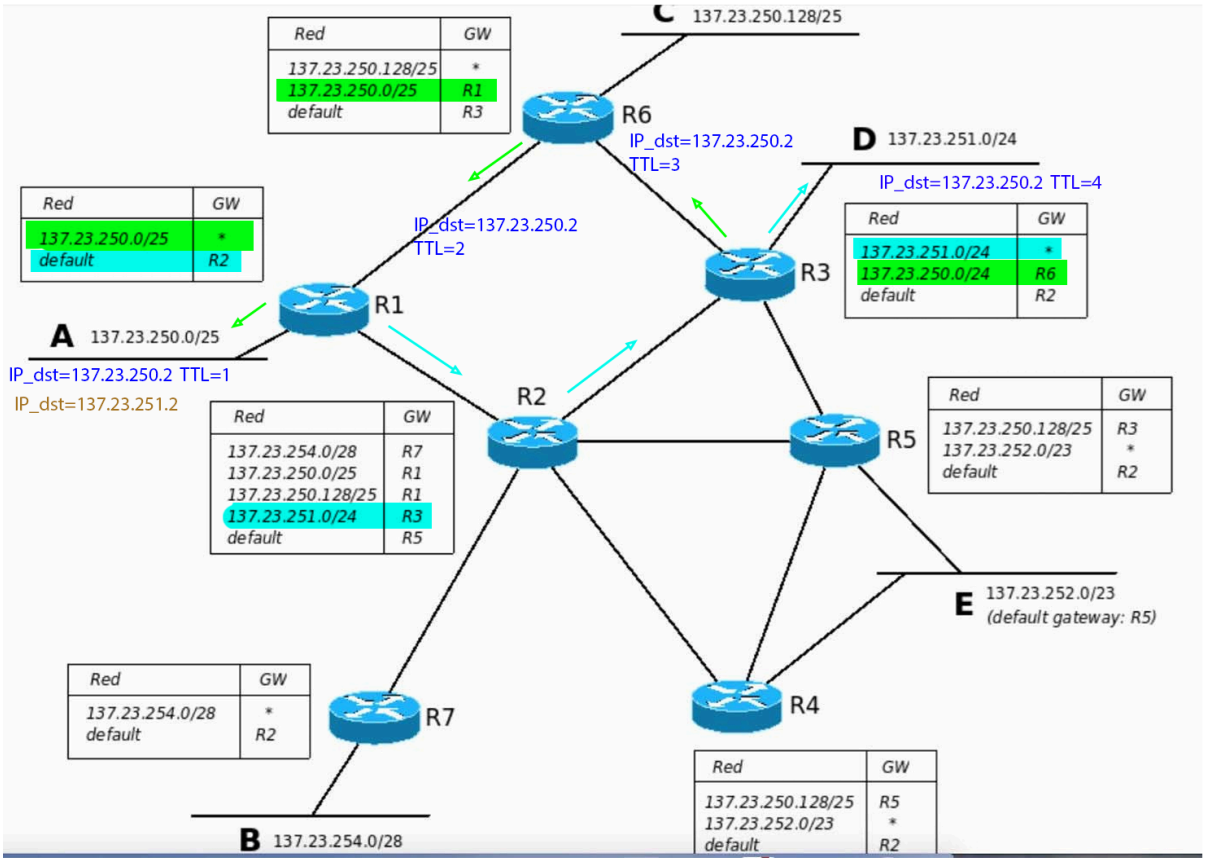

C 137.23.250.128/25
R6
IP_dst=137.23.250.2
TTL=3
D 137.23.251.0/24
IP_dst=137.23.250.2 TTL=4
R3
R1
A 137.23.250.0/25
R2
R5
E 137.23.252.0/23
(default gateway: R5)
R7
R4
B 137.23.254.0/28
Red | GW
137.23.250.128/25 | *
137.23.250.0/25 | R1
default | R3
Red | GW
137.23.250.0/25 | *
default | R2
Red | GW
137.23.251.0/24 | *
137.23.250.0/24 | R6
default | R2
Red | GW
137.23.254.0/28 | R7
137.23.250.0/25 | R1
137.23.250.128/25 | R1
137.23.251.0/24 | R3
default | R5
Red | GW
137.23.250.128/25 | R3
137.23.252.0/23 | *
default | R2
Red | GW
137.23.254.0/28 | *
default | R2
Red | GW
137.23.250.128/25 | R5
137.23.252.0/23 | *
default | R2

Paso 3


C 137.23.250.128/25
R6
IP_dst=137.23.250.2
TTL=3
D 137.23.251.0/24
IP_dst=137.23.250.2 TTL=4
IP_dst=137.23.250.2
TTL=2
R3
R1
A 137.23.250.0/25
R2
R5
E 137.23.252.0/23
(default gateway: R5)
R7
R4
B 137.23.254.0/28
Red GW
137.23.250.128/25 *
137.23.250.0/25 R1
default R3
Red GW
137.23.250.0/25 *
default R2
Red GW
137.23.251.0/24 *
137.23.250.0/24 R6
default R2
Red GW
137.23.254.0/28 R7
137.23.250.0/25 R1
137.23.250.128/25 R1
137.23.251.0/24 R3
default R5
Red GW
137.23.250.128/25 R3
137.23.252.0/23 *
default R2
Red GW
137.23.254.0/28 *
default R2
Red GW
137.23.250.128/25 R5
137.23.252.0/23 *
default R2

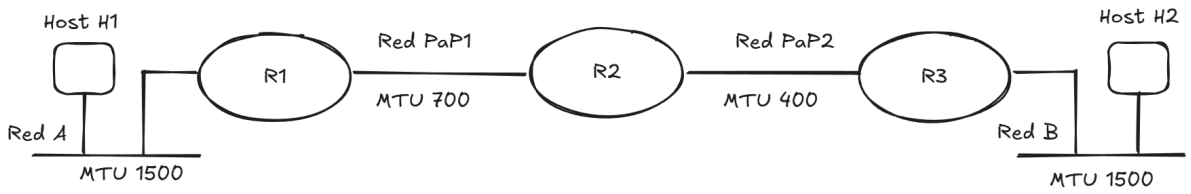

C 137.23.250.128/25
R6
IP_dst=137.23.250.2
TTL=3
D 137.23.251.0/24
IP_dst=137.23.250.2 TTL=4
IP_dst=137.23.250.2
TTL=2
R3
R1
A 137.23.250.0/25
IP_dst=137.23.250.2 TTL=1
R2
R5
E 137.23.252.0/23
(default gateway: R5)
R7
R4
B 137.23.254.0/28
Red GW
137.23.250.128/25 *
137.23.250.0/25 R1
default R3
Red GW
137.23.250.0/25 *
default R2
Red GW
137.23.251.0/24 *
137.23.250.0/24 R6
default R2
Red GW
137.23.254.0/28 R7
137.23.250.0/25 R1
137.23.250.128/25 R1
137.23.251.0/24 R3
default R5
Red GW
137.23.250.128/25 R3
137.23.252.0/23 *
default R2
Red GW
137.23.254.0/28 *
default R2
Red GW
137.23.250.128/25 R5
137.23.252.0/23 *
default R2


C 137.23.250.128/25
Red | GW
137.23.250.128/25 | *
137.23.250.0/25 | R1
default | R3
R6
IP_dst=137.23.250.2
TTL=3
D 137.23.251.0/24
IP_dst=137.23.250.2 TTL=4
Red | GW
137.23.250.0/25 | *
default | R2
IP_dst=137.23.250.2
TTL=2
Red | GW
137.23.251.0/24 | *
137.23.250.0/24 | R6
default | R2
R3
R1
A 137.23.250.0/25
IP_dst=137.23.250.2 TTL=1
IP_dst=137.23.251.2
R2
Red | GW
137.23.254.0/28 | R7
137.23.250.0/25 | R1
137.23.250.128/25 | R1
137.23.251.0/24 | R3
default | R5
R5
Red | GW
137.23.250.128/25 | R3
137.23.252.0/23 | *
default | R2
E 137.23.252.0/23
(default gateway: R5)
Red | GW
137.23.254.0/28 | *
default | R2
R7
R4
Red | GW
137.23.250.128/25 | R5
137.23.252.0/23 | *
default | R2
B 137.23.254.0/28



ICMP echo request	D(TTL=4)→R3-R6(TTL=3)→R6-R1(TTL=2)→A(TTL=1)
ICMP echo reply	A(TTL=64)→R1-R2(TTL=63)→R2-R3(TTL=62)→D(TTL=61)

Ejercicio X) En el siguiente diagrama de redes, el host H1 envía un ICMP Echo Request al host H2 con una longitud total del datagrama IP de 1500 Bytes.
a) Indique en cada red si el datagrama original se fragmentó, en caso afirmativo describa en una tabla los campos relevantes al proceso de fragmentación. En el caso de hacer suposiciones explicítelas.
b) ¿Quién reensambla?¿Por qué?
c) ¿Qué pasa con la respuesta al ICMP Echo Request? ¿Cuántos fragmentos llegan a H1?
d) Suponga que se hubiera perdido un fragmento de ICMP Echo Request, ¿qué hubiera pasado?



a) En la red A el tamaño total del datagrama coincide con el MTU (Maximum Transfer Unit) de la interfaz de red y por ende no hay fragmentación

	Total Length	MF	Fragment Offset
Red A	1500	0	0

Cuando el datagrama llega al router R1, y este ejecuta el proceso de ruteo CIDR determina que debe forwardear el datagrama por la red PaP1 que tiene un MTU 700. Como el datagrama tiene un tamaño de 1500 > 700, por lo tanto se debe adaptar el datagrama original y fragmentarlo.

Motivación

- Ajustar el tamaño del datagrama IP al MTU del enlace a atravesar.
- MTU = Maximum Data Unit
- Depende del tamaño máximo de los datos a nivel de enlace
  - para Ethernet: 1500 bytes
  - para PPPoE: 1492 bytes
  - para FDDI: 4500 bytes
- Se puede permitir o no la fragmentación. (Flag DF)

Implementación

- En encabezado original se copia en los datagramas fragmentos, y se aplican las modificaciones necesarias.
  Algunas opciones se copian en todos los fragmentos.
- Los fragmentos son datagramas independientes
- Hay un tiempo máximo de espera para los fragmentos
- Se reensambla en destino.
- Path MTU: MTU mínimo de todo el camino

Ejemplo - consideraciones

- Se fragmenta el payload original
- Los fragmentos tendrán una cantidad de datos múltiplo de 8 bytes
- El encabezado debe considerarse dentro del MTU

Suposición: vamos a suponer que el encabezado de IP es el básico de 20 Bytes.
Entonces lo que se fragmenta es el Payload de IP:

```
Payload_IP = TL - Header_IP = 1500 - 20 = 1480
```

Ahora calculamos el máximo Payload de una red con MTU de 700 Bytes siguiendo la restricción que un fragmento (que no sea el último) debe tener una cantidad de datos múltiplo de 8.

```
Payload_IP_frag = Floor((MTU - Header_IP)/8)*8
  = Floor ( (700 - 20)/8)*8 = Floor(85)*8 = 85*8 = 680
Payload_IP_frag = 680
```

El primer fragmento contendrá los primeros 680 bytes del Payload original de 1480 bytes.
1er fragmento: TL=20+680 = 700 MF=1 Offset = 0

Restan transmitir 1480 - 680 = 800 bytes
que si le sumo un header da un TL de 820 bytes que es mayor que MTU de 700, entonces no es el último fragmento.
El 2do fragmento contendrá otros 680 bytes de los 800 bytes restantes
2do fragmento: TL=20+680 = 700 MF=1 Offset = 85 (680/8)

Restan transmitir 1480 - 680 - 680 = 120 bytes
que si le sumo un header da un TL de 140 bytes que es menor que MTU de 700, por ende es el último fragmento y por ende More Fragments vale 0 .
El 3er fragmento contendrá los últimos 120 bytes
3er fragmento: TL=20+120 = 140 **MF=0** Offset = 170 ((680+680)/8)

	Total Length	**MF**	**Fragment Offset**
Red A	1500	0	0
Red PaP1 - Frag1	680	1	0
Red PaP1 - Frag2	680	1	85
Red PaP1 - Frag3	140	0	170

Cuando el datagrama llega al router R2, y este ejecuta el proceso de ruteo CIDR determina que debe forwardear el datagrama por la red PaP2 que tiene un MTU 400. Si el datagrama tiene un tamaño > 400, por lo tanto se debe adaptar el datagrama original y fragmentarlo. Este caso aplica a los primeros 2 fragmentos.

Ahora calculamos el máximo Payload de una red con MTU de 400 Bytes siguiendo la restricción que un fragmento (que no sea el último) debe tener una cantidad de datos múltiplo de 8.

```
Payload_IP_frag = Floor((MTU - Header_IP)/8)*8
  = Floor ( (400 - 20)/8)*8 = Floor(47.5)*8 = 47*8 = 376
Payload_IP_frag = 376
```

Fragmento 1 de proveniente de PaP1 (recordar TL 700, MF=1 y Foffset = 0):
El primer fragmento contendrá los primeros 376 bytes del payload del primer fragmento de 680 bytes.
1er fragmento: TL=20+376= 396 MF=1 Offset = 0

Restan transmitir 680 - 376= 304 bytes
que si le sumo un header da un TL de 324 bytes que es menor que el MTU de 400. Sin embargo como el datagrama fragmentado tiene MF=1
2do fragmento: TL=20+304 = 324 MF=1 Offset = 47 (376/8)

Fragmento 2 de proveniente de PaP1 (recordar TL 700, MF=1 y Foffset = 85):
El 3er fragmento contendrá los primeros 376 bytes del payload del segundo fragmento de 680 bytes.

3er fragmento: TL=20+376= 396 MF=1 Offset = 85

Restan transmitir 680 - 376= 304 bytes
que si le sumo un header da un TL de 324 bytes que es menor que el MTU de 400.
Sin embargo como el datagrama fragmentado tiene MF=1
4to fragmento: TL=20+304 = 324 MF=1 Offset =132 ((680+376)/8)

Fragmento 3 de proveniente de PaP1 (recordar TL 140, MF=0 y Foffset = 170):
Como TL 140 < MTU 400 no se modifica, no hay que adaptarlo al MTU de la red destino.

	Total Length	**MF**	**Fragment Offset**
Red A	1500	0	0
Red PaP1 - Frag1	680	1	0
Red PaP1 - Frag2	680	1	85
Red PaP1 - Frag3	140	0	170
Red PaP2 - Frag1.1	396	1	0
Red PaP2 - Frag1.2	324	1	47
Red PaP2 - Frag2.1	396	1	85
Red PaP2 - Frag2.2	324	1	132
Red PaP2 - Frag3	140	0	170

Observaciones:

- Bytes originalmente transmitido por H1: 1500
- 5 fragmentos recibidos por H2
- Bytes recibidos por H2: 1580 = 396+324+396+324+140 (¿A qué se deben esos 80 bytes de más?)

b) Reensambla el host destino siempre. Nunca un sistema intermediario (router) ya que no se conoce la topología y en una red donde cada paquete puede tomar un paquete independientemente un sistema intermediario podría no ver nunca todos los fragmentos. Además, no sabe si camino más adelante sería necesario volver a fragmentar con lo cual todos los recursos gastados en el router habrían sido en vano.

c) Es decir que el ICMP Echo Reply es un datagrama IP también con un total length de 1500 Bytes. Entonces, si repetimos el procedimiento hecho en a) pero con un datagrama enviado por H2 llegamos a la siguiente tabla resumen

	Total Length	**MF**	**Fragment Offset**

Red B	1500	0	0
Red PaP2 - Frag1	396	1	0
Red PaP2 - Frag2	396	1	47
Red PaP2 - Frag3	396	1	94
Red PaP2 - Frag4	372	0	141

Podemos observar que a H1 llegan 4 fragmentos a diferencia de los 5 fragmentos que hubo en el otro sentido